

1.6. Effacer à la place rangée

`texte()` et `effacer()` lisent les mêmes `x` et `y` : ils visent forcément la même case.

1. Ce qu'il faut comprendre

`effacer(x, y, MOT)` efface là où `texte(x, y, MOT)` a écrit : les deux lisent les mêmes variables.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1341 octets** de cartouche.

1. UN TEXTE NOMMÉ — MOT

```
const char MOT[] = "BONJOUR";
```

2. UNE VARIABLE — X

```
uint8_t x = 5;
```

3. UNE VARIABLE — Y

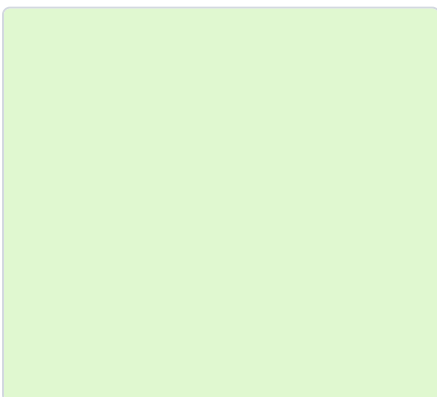
```
uint8_t y = 6;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(x, y, MOT);  
    effacer(x, y, MOT);  
  
    while (true) image();  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

« BONJOUR » est écrit puis effacé en (x, y) : l'écran reste vide.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ x vaut 5 et y vaut 6
- ✓ les sept cases de BONJOUR sont vides
- ✓ et l'écran est allumé

Ensuite : **1.7. Le mot et sa place sous un seul nom** — niveau 1 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++